A PROJECT REPORT ON

PatchScout: LLM model for vulnerability detection and generating mitigation strategies for open-source software

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

BACHELOR OF ENGINEERING
IN
COMPUTER ENGINEERING

SUBMITTED BY

Patil Parth Pawan	Exam No: B400840232
Danave Atharv Santosh	Exam No: B400840188
Malkunaik Pranav Vijay	Exam No: B400840218

Under The Guidance of
Dr. A. A. Khatri

DEPARTMENT OF COMPUTER ENGINEERING

JCEI'S JAIHIND COLLEGE OF ENGINEERING
KURAN , PINCODE 410511
SAVITRIBAI PHULE PUNE UNIVERSITY

2025-2026

CERTIFICATE

This is to certify that the project Entitled

PatchScout: LLM model for vulnerability detection and generating mitigation strategies for open-source software

Submitted by

Patil Parth Pawan	Exam No: B400840232
Danave Atharv Santosh	Exam No: B400840188
Malkunaik Pranav Vijay	Exam No: B400840218

are the bonafide student of this institute and the work has been carried out by them under the supervision of **Dr. A. A. Khatri** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of Bachelor of Engineering (Computer Engineering).

Dr. A. A. Khatri Internal Guide Dept. of Computer Engg	**Dr. A. A. Khatri** H.O.D Dept. of Computer Engg
Dr.(Mrs.) S. D. Gunjal Project Co-ordinator Dept. of Computer Engg	**Dr. D. J. Garkal** Principal Jaihind College of Engineering

PROJECT APPROVAL SHEET

PatchScout: LLM model for vulnerability detection and generating mitigation strategies for open-source software

Is successfully completed by

Patil Parth Pawan	Exam No: B400840232
Danave Atharv Santosh	Exam No: B400840188
Malkunaik Pranav Vijay	Exam No: B400840218

at

DEPARTMENT OF COMPUTER ENGINEERING

JCEI's JAIHIND COLLEGE OF ENGINEERING

SAVITRIBAI PHULE PUNE UNIVERSITY,PUNE

ACADEMIC YEAR 2025-2026

Dr. A. A. Khatri	Dr. A. A. Khatri
Internal Guide	H.O.D
Dept. of Computer Engg.	Dept. of Computer Engg.

Abstract

Open-source software (OSS) underpins modern systems but is increasingly exposed to security vulnerabilities. Traditional verification techniques such as Bounded Model Checking (BMC) provide high accuracy but are computationally expensive and impractical for real-time developer workflows. Large Language Models (LLMs) fine-tuned for code, exemplified by SecureFalcon, provide a promising middle ground delivering rapid inference with high classification performance. This review synthesizes recent advances in LLM-based vulnerability detection and closely examines challenges related to dataset realism, input representation, and overfitting (highlighted by Real-Vul and EPVD). We further survey verification and repair techniques TransferFuzz-Pro for LLM-assisted verification and DCAFixer for automated patch generation—to outline a pathway toward end-to-end automated vulnerability management in OSS ecosystems.

keywords:- Large Language Models, Vulnerability Detection, Open-Source Software, SecureFalcon, Automated Repair , etc.

Acknowledgments

With immense pleasure, we present the Project report as part of the curriculum of the B.E Computer Engineering. We wish to thank and express deep sense of gratitude to our guide **Dr. A. A. Khatri** for his guidance, inspiration in project work.

We also wish to thank our Project Coordinator **Dr. S. D. Gunjal** for encouragement and he patiently discussed ideas with us and gave indispensable suggestions.

We are grateful to **Dr. A. A. Khatri**, Head of Department of Computer Engineering for always being ready to help with the most diverse problems that we have encountered along the way.

Patil Parth Pawan	Exam No: B400840232
Danave Atharv Santosh	Exam No: B400840188
Malkunaik Pranav Vijay	Exam No: B400840218

List of Figures

5.1 Waterfall Model 21
5.2 Time Line Chart 24

6.1 Use Case Diagram 29
6.2 DFD 0 Diagrams 31
6.3 DFD 1 Diagrams 32
6.4 Activity diagram 34
6.5 Sequence Diagram 36

7.1 Architecture Diagram of PatchScout System 41
7.2 Class Diagram 42

A.1 IDEA Matrix 60

B.1 Divide and Conquer diagram 62
B.2 Venn diagram for the Computational Complexity Classes 65

E.1 Plagiarism Report 69

INDEX

1 Synopsis **1**

1.1 Project Title 1

1.2 Project Option 1

1.3 Internal Guide 1

1.4 Technical Keywords(As per ACM keywords) 1

1.5 Problem Statement 2

1.6 Abstract 2

1.7 Goals and Objectives 2

1.8 Relevant mathematics associated with the Project 3

1.9 Names of Conferences / Journals where papers published 4

1.10 Review of Conference/Journal Papers supporting Project idea 5

1.11 Plan of Project Execution 6

2 Technical Keywords **7**

2.1 Area of Project 7

2.1.1 Technical Keywords 7

3 INTRODUCTION **8**

3.1 Project Idea 8

3.2 Motivation 9

3.3 Literature Survey 9

4 Problem Definition and scope **13**

4.1 Problem Statement 13

4.1.1 Goals and objectives 13

4.1.2 Statement of scope 14
4.2 Major Constraints 15
4.3 Methodologies of Problem solving and efficiency issues 15
4.4 Outcome 16
4.5 Applications 17
4.6 Hardware Requirements 18
4.7 Software Requirements (Platform Choice) 19

5 Project Plan 20
5.1 Project Estimates 20
5.1.1 Project Description: 22
5.2 Risk Management 22
5.2.1 Risk Identification 22
5.2.2 Risk analysis 23
5.2.3 Overview of Risk Mitigation, Monitoring, Management . . 24
5.3 Project Schedule 24
5.3.1 Time Line Chart 24
5.4 Team Organization 25
5.4.1 Team Structure 25

6 Software Requirement Specification 26
6.1 Introduction 26
6.1.1 Purpose and Scope of Document 26
6.1.2 Overview of responsibilities of Developer 27
6.2 Usage Scenario 27
6.2.1 User profiles 27
6.2.2 Use Case View 28
6.3 Data Model and Description 30
6.3.1 Data Description 30
6.3.2 Data objects and Relationships 30
6.4 Functional Model and Description 31
6.4.1 Data Flow Diagram 31

6.4.2 Non Functional Requirements 32
6.4.3 Description of functions 33
6.4.4 Activity Diagram 33
6.4.5 Design Constraints 35
6.4.6 Software Interface Description 35
6.4.7 Sequence Diagram 36

7 Detailed Design Document using Appendix A and B 37
7.1 Introduction 37
7.1.1 Data Flow Summary 38
7.2 Architectural Design 38
7.3 Component Design 41
7.3.1 Class Diagram 42

8 Project Implementation 43
8.1 Introduction 43
8.2 Tools and Technologies Used 45
8.2.1 Python 45
8.3 Methodologies/Algorithm Details 48
8.3.1 PatchScout Vulnerability Detection Workflow 48
8.3.2 Pseudo Code for PatchScout Hybrid Classification 48
8.4 Workflow Integration 49
8.5 Verification and Validation for Acceptance 49

9 Software Testing 51
9.1 Test Plans 51
9.2 Testing Procedure 51
9.3 Test Strategy 52
9.3.1 Unit Testing 52
9.3.2 Integration Testing 53
9.3.3 Performance Testing 53
9.3.4 Regression Testing 53

10 Conclusion and future scope **54**

10.1 Conclusion 54

10.2 Future Scope 54

11 REFERENCES **56**

A Laboratory assignments on Project Analysis of Algorithmic Design **57**

B Laboratory assignments on Project Quality and Reliability Testing of Project Design **61**

B.1 COMPUTATIONAL COMPLEXITY CLASSES THEORY 64

B.2 Why PatchScout Prioritizes P-Class Feasibility 65

B.3 Conclusion: 65

C Project Planner **66**

D Reviewers Comments of Paper Submitted **68**

E Plagiarism Report **69**

F Information of Project Group Members **70**

Chapter 1

Synopsis

1.1 PROJECT TITLE

PatchScout: LLM model for vulnerability detection and generating mitigation strategies for open-source software

1.2 PROJECT OPTION

Internal project

1.3 INTERNAL GUIDE

Dr. A. A. Khatri

1.4 TECHNICAL KEYWORDS(AS PER ACM KEYWORDS)

1. Machine learning for code analysis
2. Software security
3. Static analysis
4. Code embeddings
5. Vulnerability detection
6. Large Language Models (LLMs)

7. Abstract Syntax Tree (AST) Parsing

8. OWASP/CWE Mapping

1.5 PROBLEM STATEMENT

To Develop an explainable AI-driven tool leveraging LLMs and related techniques to detect vulnerabilities and malicious code in software source code with a focus on open-source projects and provide CWE-based classification with actionable mitigation measures.

1.6 ABSTRACT

Open-source software (OSS) underpins modern systems but is increasingly exposed to security vulnerabilities. Traditional verification techniques such as Bounded Model Checking (BMC) provide high accuracy but are computationally expensive and impractical for real-time developer workflows. Large Language Models (LLMs) fine-tuned for code, exemplified by SecureFalcon, provide a promising middle ground delivering rapid inference with high classification performance. This review synthesizes recent advances in LLM-based vulnerability detection and closely examines challenges related to dataset realism, input representation, and overfitting (highlighted by Real-Vul and EPVD). We further survey verification and repair techniques TransferFuzz-Pro for LLM-assisted verification and DCAFixer for automated patch generation—to outline a pathway toward end-to-end automated vulnerability management in OSS ecosystems.

1.7 GOALS AND OBJECTIVES

The key objectives of this project are:

1. To study existing research and real-world vulnerability datasets to identify high-impact CWEs and analyze limitations of current static and ML-based detection methods.

2. To design and implement a hybrid vulnerability detection system that integrates static analysis with ML/LLM-based code understanding for improved precision and explainability.

3. To generate explainable reports with line-level evidence and mitigation suggestions.

4. To evaluate and compare the proposed system's performance (precision, recall, and F1-score) against traditional static analysis tools such as SpotBugs and Bandit.

1.8 RELEVANT MATHEMATICS ASSOCIATED WITH THE PROJECT

System Description:

- Let S be the Whole system S= I,P,O
- I : Input
- P : Procedure
- O : Output
- Input(I)
- I= dataset[Source Code Samples]
- where Dataset [Source Code] refers to a curated set (e.g., SARD, BigVul, and user-uploaded Java/Python files)
- Procedure (P)=P: Accepts input code, runs static analysis, parses ASTs, embeds code paths, decides vulnerability status, and generates mitigation.
- Space Complexity: Space complexity is dominated by code embedding and AST storage:

  For N files,M functions, and K AST paths per function, space O(N×M×K×D), where D = embedding vector size (e.g., 768 for CodeBERT).

- Time Complexity: Time Complexity Analysis

  - Static Analysis: O(N) — linear with the number of files.

  - AST Parsing: O(M × T) — where T is the average parsing time per function.

  - CodeBERT Embedding: O(P) — where P is the number of code paths analyzed.

  - Ensemble Decision / Report Generation: O(M).

  - Overall Time Complexity: O(N + M × T + P) per project — scales with dataset size and code complexity.

  For large projects or datasets, both runtime and memory usage increase accordingly.

- Output (O): O = Vulnerability Report, Mitigation Suggestions

  Reports are generated in JSON, PDF, or HTML formats and include:

  File name, Line numbers, CWE classification, Explanation tokens

- Failures: Large datasets or massive codebases may increase runtime and memory demand, possibly leading to timeouts or incomplete analysis. Incomplete AST extraction if code is poorly structured, affecting accuracy.

- Success:
  - System accurately detects vulnerabilities, pinpoints their location, and generates clear fix templates.
  - User easily interprets explanations and mitigation steps to secure their code with minimal effort.

1.9 NAMES OF CONFERENCES / JOURNALS WHERE PAPERS PUBLISHED

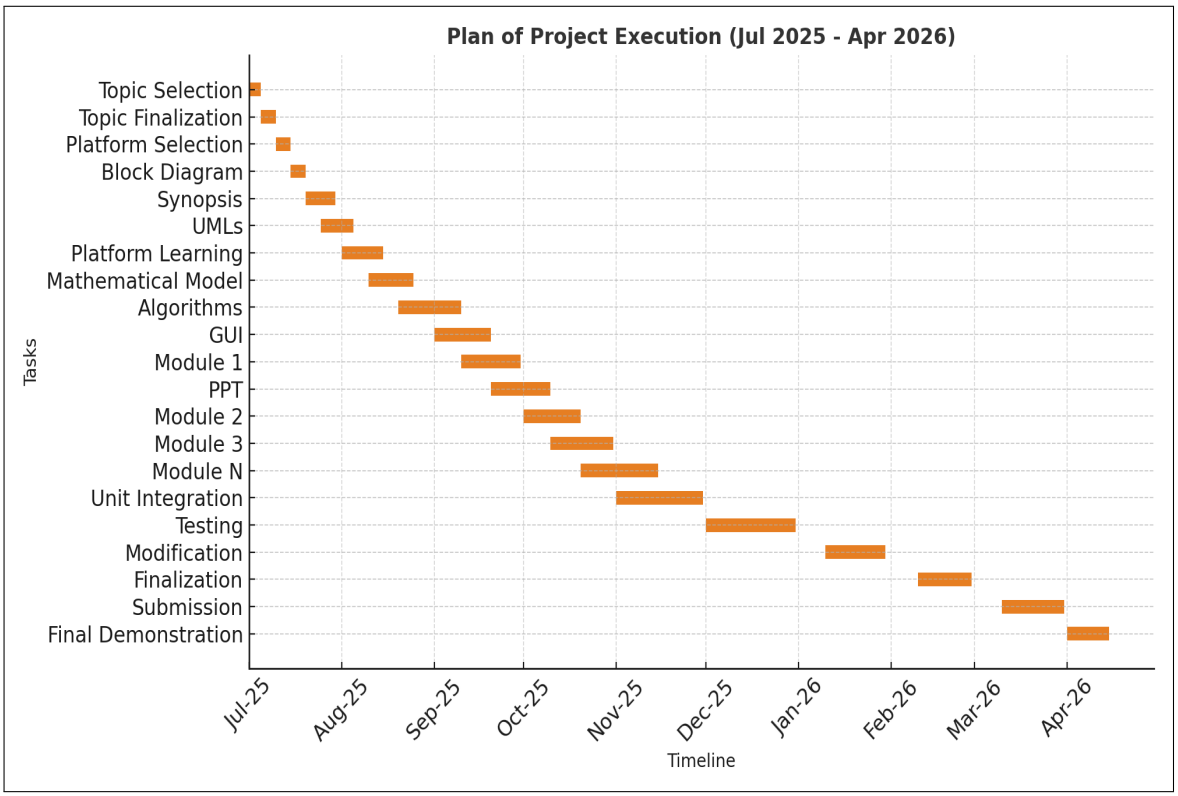
1. International Journal of Sciences and Innovation Engineering (IJSCI)

1.10 REVIEW OF CONFERENCE/JOURNAL PAPERS SUPPORTING PROJECT IDEA

(a) Mohamed Amine Ferrag, Ammar Battah, Norbert Tihanyi, Ridhi Jain, Diana Maimuţ, Fatima Alwahedi, Thierry Lestable, Narinderjit Singh Thandi, Abdechakour Mechri, Merouane Debbah, and Lucas C. Cordeiro, "**SecureFalcon: Are We There Yet in Automated Software Vulnerability Detection With LLMs?"**,IEEE Transactions on Software Engineering, Vol. 2025, Mar. 2025.

This study addresses the challenge of achieving both speed and accuracy in software vulnerability detection. While formal verification methods like Bounded Model Checking ensure safety by eliminating false positives, they are too slow for real-time applications. To resolve this, the authors propose SecureFalcon, a lightweight Large Language Model with only 121 million parameters, derived from the Falcon-40B architecture and optimized for vulnerability classification. Trained on two high-quality datasets—FormAI, containing ESBMC-verified C code, and FalconVulnDB, which aggregates sources like SySeVR, BigVul, and ReVeal to cover the Top 25 CWEs—SecureFalcon achieves 94% accuracy in binary and 92% in multiclass classification for C/C++ code. It outperforms models such as CodeBERT, BERT, and RoBERTa by about 4%, offering near-instant predictions and making it ideal for fast, accurate, real-time vulnerability detection in modern software development workflows.

1.11 PLAN OF PROJECT EXECUTION



Chapter 2

Technical Keywords

2.1 AREA OF PROJECT

Source Code Security and Machine Learning

2.1.1 Technical Keywords

(a) Machine learning for code analysis

(b) Software security

(c) Static analysis

(d) Code embeddings

(e) Vulnerability detection

(f) Large Language Models (LLMs)

(g) Abstract Syntax Tree (AST) Parsing

(h) OWASP/CWE Mapping

(i) Secure Software Engineering

Chapter 3

INTRODUCTION

3.1 PROJECT IDEA

Modern software development employs millions of lines of code and widespread reuse of third-party libraries, making reliable security analysis essential. Vulnerabilities hidden in code such as injection flaws, hard-coded secrets, or improper input handling can have critical real-world impact. Relying on traditional rule-based static analysis tools alone is insufficient, as these tools frequently generate too many false positives and lack semantic understanding of complex code.

With advances in artificial intelligence and machine learning, language models pre-trained on code (such as CodeBERT) have shown promise in understanding nuanced programming patterns. This project, PatchScout, proposes a hybrid system that combines static analysis with machine learning, leveraging the power of Abstract Syntax Tree (AST) execution paths and transformer-based models to automatically detect vulnerabilities and provide actionable, context-specific mitigation recommendations. The aim is to make developer-centric vulnerability reporting both more accurate and much more usable.

3.2 MOTIVATION

With the growing reliance on open-source code and software supply chains, a single undetected vulnerability can spread across thousands of systems, causing severe security risks. Traditional rule-based detection methods often lack contextual understanding, leading to excessive false alerts. There is a strong need for tools that not only detect vulnerabilities accurately but also explain their findings in a clear and actionable way. PatchScout addresses this by combining static analysis for rule-based insights with ML-based code understanding for deeper semantic reasoning, reducing false positives and enabling developers to focus on real threats through precise, line-level reports and practical mitigation suggestions.

3.3 LITERATURE SURVEY

Automated vulnerability detection has become increasingly critical as software systems grow in complexity and dependency on open-source components. Traditional code analysis methods, often reliant on fixed rules and handcrafted patterns, tend to generate excessive false positives and lack the ability to interpret context-rich code constructs. The rapid evolution of Artificial Intelligence (AI) and the introduction of deep language models specifically trained on source code have enabled significant advances in this domain. Recent research now emphasizes the integration of static analysis techniques with machine learning-based models, leveraging the nuanced understanding of code semantics that modern AI provides. This literature survey examines how these hybrid and learning-based approaches are shaping the future of vulnerability detection—leading to systems that are not only more accurate and context-aware but also capable of providing developers with interpretable, actionable insights for secure software development.

1. SecureFalcon: Are We There Yet in Automated Software Vulnerability Detection With LLMs?

Authors: Ferrag, M. A., et al. (2025)

Summary: This paper presents SecureFalcon, an innovative and compact LLM (121M parameters) derived from Falcon-40B, designed for software vulnerability classification. It aims to combine the high accuracy of Formal Verification (FV) with the real-time inference speed needed for code completion frameworks. SecureFalcon was fine-tuned on the FormAI dataset (AI-generated and ESBMC-verified) and FalconVulnDB, which aggregates datasets like BigVul and ReVeal, covering the top 25 critical CWEs. The model achieved 94% accuracy in binary and 92% in multiclass classification, outperforming BERT and CodeBERT by 4%. Ablation studies showed that positional embeddings and domain-specific tokens were crucial for maintaining strong performance.

Relevance: This work is significant as it introduces a compact LLM optimized for fast, accurate vulnerability detection, enabling practical use in open-source development. Trained on aggregated datasets like FalconVulnDB, it validates the effectiveness of LLMs for high-performance cybersecurity applications.

2. Multitask-Based Evaluation of Open-Source LLM on Software Vulnerability

Authors: Yin, X., Ni, C., Wang, S. (2024)

Summary: This study evaluates LLMs on four vulnerability tasks—detection, assessment, location, and description—using the Big-Vul dataset. Compared to transformers, fine-tuned LLMs like DeepSeek-Coder 6.7B excel in assessment with added context but lag in detection efficiency. For description tasks, trimming verbose outputs (e.g., CodeLlama-Trim) significantly improved performance, nearing CodeT5+ results.

Relevance: This paper highlights that domain-specific fine-tuning is crucial for LLMs in vulnerability detection, aligning closely with the project's Big-

Vul-based focus.

3.Revisiting the Performance of Deep Learning-Based Vulnerability Detection on Realistic Datasets

Authors: Chakraborty, P., Arumugam, K. K., Alfadel, M., Nagappan, M., McIntosh, S. (2024)

Summary: This research introduces Real-Vul, a realistic open-source dataset exposing major reliability gaps in deep learning-based vulnerability detectors. Models like DeepWukong and ReVeal showed up to a 95% precision drop due to false positives and overlapping embeddings. Data augmentation (e.g., dead code insertion) improved performance by up to 30%, highlighting overfitting issues in existing models.

Relevance: This work stresses realistic LLM evaluation, showing that general models like CodeLlama and Mixtral underperform on real codebases (e.g., Chromium, Linux) compared to specialized tools like LineVul, highlighting the need for task-specific optimization.

4. TransferFuzz-Pro: Large Language Model Driven Code Debugging Technology for Verifying Propagated Vulnerability

Authors: Li, S., Xie, K., Li, Y., Li, H., Ren, Y., Sun, L., Zhu, H. (2025)

Summary: TransferFuzz-Pro is an automated fuzzing framework for verifying propagated vulnerabilities in C/C++ binaries using LLM-driven code debugging to automate option generation for directed fuzzing. It achieves up to 26.2× faster verification than state-of-the-art methods and can generate PoCs for Linux binaries without source code through binary-level instrumentation.

Relevance: This paper highlights LLMs in advanced verification roles beyond detection, showcasing their ability to automate complex debugging for open-source code reuse.

5. mVulPreter: A Multi-Granularity Vulnerability Detection System with Interpretations

Authors: Zou, D., Hu, Y., Li, W., Wu, Y., Zhao, H., Jin, H. (2022)

Summary: mVulPreter is a multi-granularity DL detector combining function- and slice-level models for improved accuracy. Using slice purification and attention-based interpretation, it pinpoints vulnerable statements and outperforms seven SOTA methods, discovering 28 new vulnerabilities in large open-source codebases.

Relevance: This work sets a benchmark for accurate, scalable, and self-interpretable analysis by using model-based attention and probabilities, highlighting the value of multi-granularity analysis over the less specific outputs of general LLMs.

6. HeVulD: A Static Vulnerability Detection Method Using Heterogeneous Graph Code Representation

Authors: Huang, Y., et al. (2024)

Summary: HeVulD is a C/C++ static vulnerability detector using heterogeneous graphs for precise code representation and key-node slicing to cut computation. It achieved a 96.4% F1-score on SARD with strong multi-class detection, noting that most scanned codebases rely on open-source components.

Relevance: This showcases that high-performance DL requires complex graph representations (GNNs) for C/C++ vulnerability detection in open source, setting a significant benchmark for LLM inputs.

The reviewed research highlights rapid progress in automated software vulnerability detection through the integration of deep learning and extensive, diverse code datasets. These works consistently underscore the value of high-quality annotated datasets, the need for scalable evaluation frameworks, and the effectiveness of hybrid models for real-world code analysis.

Chapter 4

Problem Definition and scope

4.1 PROBLEM STATEMENT

- To Develop an explainable AI-driven tool leveraging LLMs and related techniques to detect vulnerabilities and malicious code in software source code with a focus on open-source projects and provide CWE-based classification with actionable mitiga- tion measures.

In the rapidly evolving software development landscape, vulnerabilities hidden within source code pose significant security risks. Traditional static analysis tools often generate high false positives and lack the contextual awareness necessary for precise vulnerability identification. As organizations increasingly rely on open-source software, there is an urgent need for automated, accurate, and scalable vulnerability detection systems that provide explainable and actionable insights to developers. PatchScout addresses this gap by combining static analysis with large language model-based semantic understanding to detect vulnerabilities, localize them at the code line level, and offer effective mitigation strategies.

4.1.1 Goals and objectives

The key objectives of this project are:

(a) To study existing research and real-world vulnerability datasets to identify high-impact CWEs and analyze limitations of current static and ML-based detection methods.

(b) To design and implement a hybrid vulnerability detection system that integrates static analysis with ML/LLM-based code understanding for improved precision and explainability.

(c) To generate explainable reports with line-level evidence and mitigation suggestions.

(d) To evaluate and compare the proposed system's performance (precision, recall and F1-score) against traditional static analysis tools such as SpotBugs and Bandit.

4.1.2 Statement of scope

This project is designed to:

- Develop a scalable and modular hybrid framework that integrates static code analysis and machine learning techniques for effective vulnerability detection in open-source projects.
- Implement an advanced transformer-based model utilizing AST-based execution path embeddings to capture semantic code context for vulnerability classification.
- Provide real-time vulnerability detection, token-level explanatory insights, and CWE-aligned mitigation templates to assist developers in secure code maintenance.
- Enable future extensions with multi-language support, richer vulnerability categories, and integration with CI/CD pipelines for continuous security assessment.

4.2 MAJOR CONSTRAINTS

- **Language Support:**
  - **C**urrently, the system supports only Java and Python language code bases, limiting its applicability to projects mainly written in these languages.
- **Scalability:**
  - **T**he system is optimized to handle small to medium-sized open-source projects, generally up to 100,000 lines of code; processing very large codebases may require additional optimization.
- **Model Limitations:**
  - The **T**he machine learning components may produce false negatives when confronted with novel vulnerability patterns or heavily obfuscated code, as such cases may not be well represented in training data.
- **Resource Consumption:**
  - The **D**ue to the hybrid nature of static and ML-based analysis, the system requires moderate CPU and memory resources, with performance depending on the input project size and complexity.

4.3 METHODOLOGIES OF PROBLEM SOLVING AND EFFICIENCY ISSUES

(a) **Requirement gathering and analysis:** This phase involves identifying the hardware, software, datasets, and interface requirements necessary for PatchScout. Stakeholder interviews and analysis define the feature set and security goals, which are documented clearly as system specifications.

(b) **System Design:**Based on requirements, the system's architecture and workflow are designed for clarity and usability. UML diagrams including class diagrams, sequence diagrams, and data flow diagrams (DFDs)

are created to model system components, interactions, and process flows to ensure smooth implementation and testing.

(c) **Implementation:** The project is divided into smaller functional modules such as static analysis, AST parsing, ML embedding, ensemble decision, explanation, and report generation. Each module is developed as an independent unit. Unit testing validates correctness and functionality of each module before integration.

(d) **Testing:** After unit completion, integration testing is conducted to ensure that modules work seamlessly together. Various test cases mimic real-world scenarios to verify the detection accuracy, explanation relevance, and report comprehensiveness within acceptable performance constraints.

(e) **Deployment of System:** Upon successful testing, PatchScout is deployed either as a cloud service or integrated into developer pipelines. The deployment includes configuring user access, enabling report generation endpoints, and ensuring scalability under load.

(f) **Maintenance:** There are some issues which come up in the client environment. To fix those issues patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment. All these phases are cascaded to each other in which progress is seen as flowing steadily downwards like a waterfall through the phases. The next phase is started only after the defined set of goals are achieved for previous phase and it is signed off, so the name ”Waterfall Model”. In this model phases do not overlap.

4.4 OUTCOME

- Provide high accuracy in detecting software vulnerabilities to reduce false positives and negatives.

- Deliver detailed, token-level explanations to facilitate developer understanding and faster remediation.
- Save time by automating vulnerability detection workflows integrated into developer environments.
- Enhance secure coding practices through actionable, CWE-aligned mitigation templates.

4.5 APPLICATIONS

The PatchScout system offers a range of practical applications that enhance software security and developer productivity. Key applications include:

1. Continuous Integration and Continuous Deployment (CI/CD) Pipelines

PatchScout can be integrated into CI/CD workflows to provide automated, real-time vulnerability scanning during build and deployment phases, ensuring that security issues are caught early in the development cycle.

2. Developer Assistance and Education

The system provides real-time feedback within integrated development environments (IDEs) via plugins, helping developers understand and fix vulnerabilities as they code. The explainable output facilitates learning and adoption of secure coding practices.

3. Security Audit and Compliance

Security teams can leverage PatchScout to generate detailed, interpretable reports for auditing and compliance verification, reducing manual effort while increasing accuracy.

4. Vulnerability Triage and Patch Prioritization

By offering confidence scoring and in-depth explanations, PatchScout aids security analysts in prioritizing vulnerability remediation efforts, accelerating secure software delivery.

5. Runtime Monitoring Extensions

PatchScout sets the foundation for future integration with runtime application self-protection (RASP) and dynamic vulnerability detection, broadening the defense mechanisms beyond static analysis.

6. Strengthening Open-Source Ecosystems

Open-source maintainers can incorporate PatchScout to continuously monitor and improve code quality and security, thereby reinforcing trust in widely used software components.

7. Educational and Training Tools

PatchScout's transparent detection and mitigation guidance can be employed in educational settings to train new developers and security professionals on vulnerability patterns and fixes, fostering a security-aware development culture.

4.6 HARDWARE REQUIREMENTS

- **Processor:**
  Minimum Intel Core i3 or equivalent, equipped with multi-core architecture to support parallel processing for static and ML-based vulnerability analysis.

- **RAM:**
  At least 8 GB to handle memory-intensive AST parsing, embedding generation, and ML inference tasks during scanning.
- **Storage:**
  Minimum 50 GB of free disk space to accommodate source code repositories, model checkpoints (e.g., CodeBERT weights), temporary caches, and report storage.
- **Optional GPU:**
  Recommended for improving the speed of transformer-based model inference and training, especially for larger datasets or multiple concurrent analyses.

4.7 SOFTWARE REQUIREMENTS (PLATFORM CHOICE)

- IDE: Visual Studio Code or IntelliJ IDEA recommended for seamless integration and code management.
- Programming Languages: Python (3.8 or higher) for ML pipelines and orchestration; Java (8 or higher) for static analysis integration.
- Core Libraries and Frameworks: HuggingFace Transformers for ML models, TensorFlow or PyTorch as ML backends, Tree-sitter for AST parsing, SpotBugs and Bandit for static analysis engines.
- Databases: PostgreSQL or SQLite for storing mitigation templates, CWE mappings, and tracking vulnerabilities.
- Web Framework: Flask or FastAPI for building the user interface and API endpoints to support web-based interactions.

Chapter 5

Project Plan

5.1 PROJECT ESTIMATES

The Waterfall Model was first Process Model to be introduced. It is also referred to as a linear-sequential life cycle model. It is very simple to understand and use. In a waterfall model, each phase must be completed fully before the next phase can begin. This type of model is basically used for the for the project which is small and there are no uncertain requirements. At the end of each phase, a review takes place to determine if the project is on the right path and whether or not to continue or discard the project. In this model the testing starts only after the development is complete. In waterfall model phases do not overlap.

(a) **Requirement Gathering and Analysis:**
All system and security requirements for PatchScout are identified in this phase, including software and hardware prerequisites, integration points, required datasets, and reporting needs.

(b) **System Design:** Comprehensive architecture diagrams, UML class diagrams, and data flow diagrams are developed to map out module relationships and information flows. This phase ensures each function and user interaction is precisely defined.

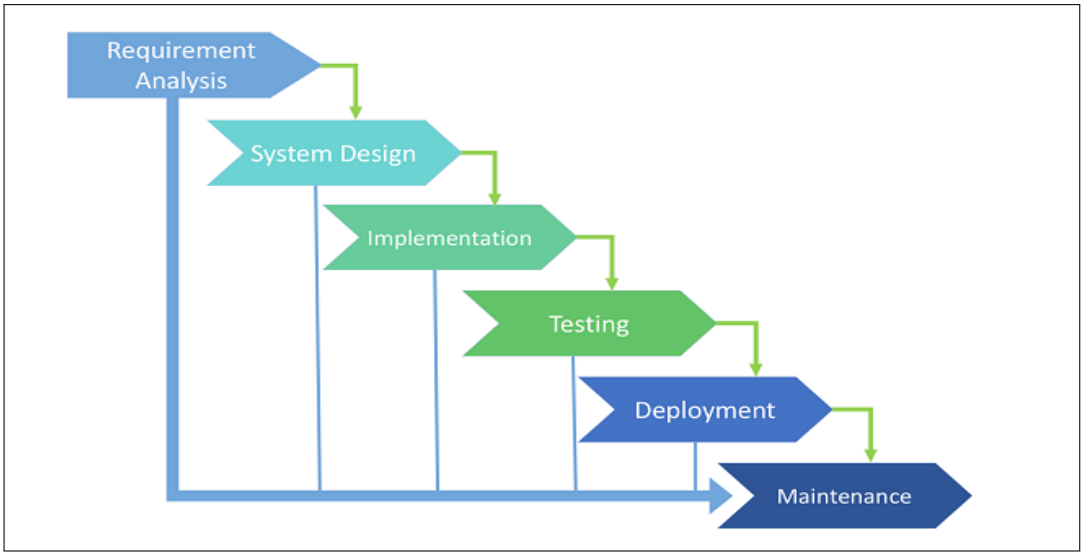


Figure 5.1: Waterfall Model

(c) **Development and programming** Following design approval, development begins. Machine learning pipelines (using Python) and static analysis integration (using both Python and Java components) are implemented.

- Source code writing
- Module-level testing and debugging

(d) **Testing:** All PatchScout modules (static analyzers, ML engines, reporting dashboards) undergo extensive test cases. Integration testing validates that end-to-end workflow performs as specified, with a focus on vulnerability detection accuracy, report explainability, and performance.

(e) **Deployment:** Once validated, PatchScout is deployed for real-world developer and security team use, with integration into CI/CD pipelines or as a standalone web platform.

5.1.1 Project Description:

Phase	Task	Description
Phase 1	Analysis	Review LLMs and static tool integration best practices.
Phase 2	Literature survey	Summarize related research on hybrid vulnerability detection.
Phase 3	Design	Define module interactions, explainability requirements, UIs
Phase 4	Implementation	Code static rules, ML models, interfaces, reporting features.
Phase 5	Testing	Evaluate performance, accuracy, usability.
Phase 6	Documentation	Prepare user guides, technical and research documentation.

Table 5.1: Project Scheduling

5.2 RISK MANAGEMENT

5.2.1 Risk Identification

Technological Risks

- ML model limitations—novel vulnerability patterns may be undetected
- Static analyzer rule drift—patterns may require continual update
- Integration bugs—cross-language module failures, data exchange mismatches.

Environmental Risks

- Server downtime or resource bottlenecks impact scanning performance
- Third-party dependency updates may break existing integrations

Operational Risks

- Delays in open-source data/model availability hinder continuous improvement.
- Over-reliance on a specific static analysis tool leads to coverage gaps.

Data and Security Risks

- Privacy concerns around storing scanned source code (may require anonymization)
- False positives/negatives impact developer trust and system adoption

5.2.2 Risk analysis

Continuous evaluation and mitigation of identified risks is integral to the project. Examples:

- Routine retraining of models with new vulnerability samples to address ML drift
- System monitoring to address integration or performance bottlenecks.
- Version-pinning and frequent testing of third-party dependencies

5.2.3 Overview of Risk Mitigation, Monitoring, Management

Risk management organizational role:

Risk management in PatchScout is collaborative.

- The development team leads ongoing technical risk monitoring, regression testing, and dependency upkeep.
- Project leads and all contributors are empowered to flag emerging risks (technical, process, integration, or data security) for expedient review.
- Shared documentation and transparent communication ensure that both anticipated and unforeseen risks are managed proactively and iteratively.

5.3 PROJECT SCHEDULE

5.3.1 Time Line Chart

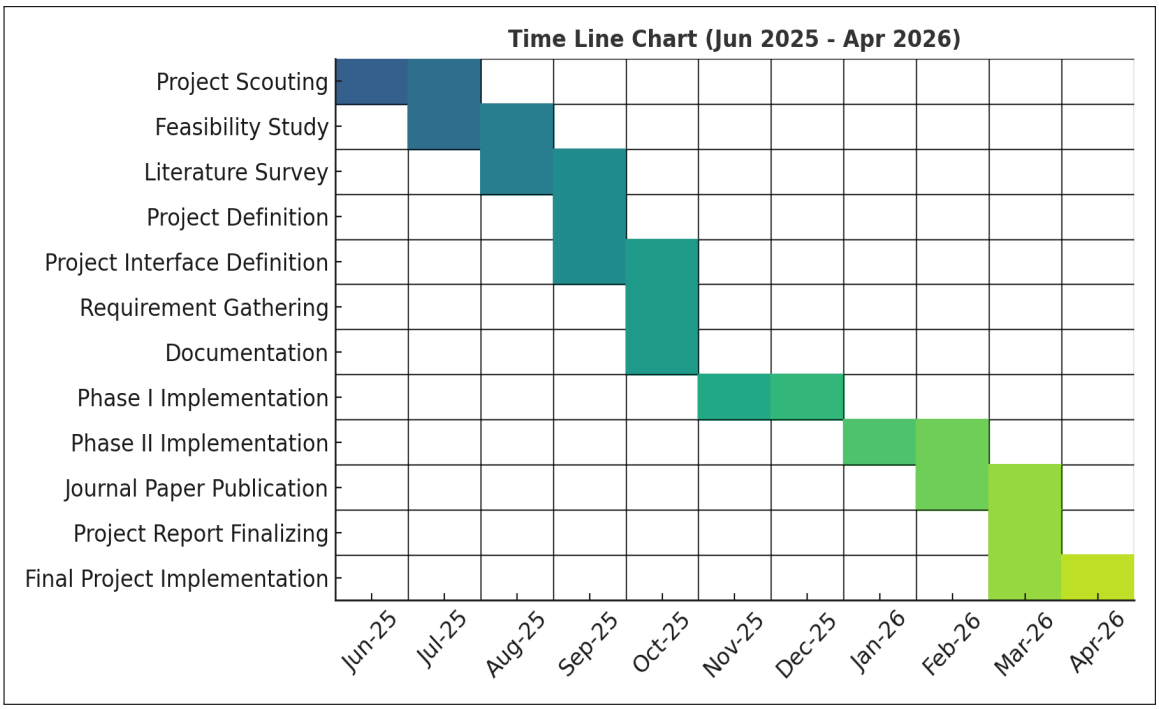


Figure 5.2: Time Line Chart

5.4 TEAM ORGANIZATION

5.4.1 Team Structure

Name	**Role**
Patil parth Pawan	Coding
Danave Atharv Santosh	Designing and Documentation
Maluknaik Pranav Vijay	Testing module

Table 5.2: Team Structure

Chapter 6

Software Requirement Specification

6.1 INTRODUCTION

This Software Requirements Specification (SRS) defines a structured foundation for implementing PatchScout, a scalable vulnerability detection and mitigation system using Large Language Models (LLMs). It outlines the scope, system usage context, data flows, constraints, and interface expectations critical for the successful development and deployment of the software.

6.1.1 Purpose and Scope of Document

This SRS document provides a detailed description of functional and non-functional requirements for the PatchScout project—an LLM-based hybrid vulnerability detection framework designed for Java and Python open-source projects. The system aims to enable accurate detection, token-level explanations, and automated mitigation suggestion generation to assist developers and security analysts in improving code security.

The scope includes static analysis integration, syntax-driven ML embeddings, vulnerability classification, explanation extraction, mitigation template management, and comprehensive reporting.

6.1.2 Overview of responsibilities of Developer

The development team is responsible for:

- Designing and implementing a secure web-based system and RESTful APIs supporting vulnerability scan requests and result retrieval.
- Developing and fine-tuning LLM models (CodeBERT-based) for effective vulnerability classification.
- Integrating static analyzers and AST parsing for semantic code understanding.
- Creating interfaces for vulnerability explanation and mitigation recommendation generation.
- Building a responsive user interface for code upload, scan monitoring, and result visualization.
- Ensuring data pipeline reliability supporting batch and real-time processing.

6.2 USAGE SCENARIO

This section provides various usage scenarios for the system to be developed

6.2.1 User profiles

- **Developers:** Upload source code or repositories for vulnerability scanning; receive detailed reports to remediate security issues.
- **Security Analysts:** Use PatchScout to prioritize vulnerability risks based on classification confidence and explanations for expedited patching.
- **DevOps Engineers:** Integrate PatchScout scanning into CI/CD pipelines for automated security validation during software delivery.

- **Project Managers:** Access vulnerability metrics and reports to monitor overall software security health and compliance status.
- **Core Maintainers:** Annotate datasets and contribute to model retraining with new vulnerability insights for continual system improvement.

6.2.2 Use Case View

UML use case diagram are used to describe the functionality of particular system. The following shows the functionality of the user of the system.

- **UC1: Code Upload and Validation** — Users can submit Programming language code files or repositories for analysis.
- **UC2: Hybrid Vulnerability Scan Execution** — The system runs static analysis alongside ML-based semantic vulnerability detection.
- **UC3: Explanation and Mitigation Suggestion** — Generate token-level explanations and CWE-aligned remediation templates for each detected vulnerability.
- **UC4: Dataset and Model Management** — Administrators manage annotated data and trigger retraining of ML models.
- **UC5: Reporting and Exporting** — Generate detailed reports in multiple formats (JSON, PDF, HTML) summarizing vulnerabilities, confidence scores, and fix suggestions.

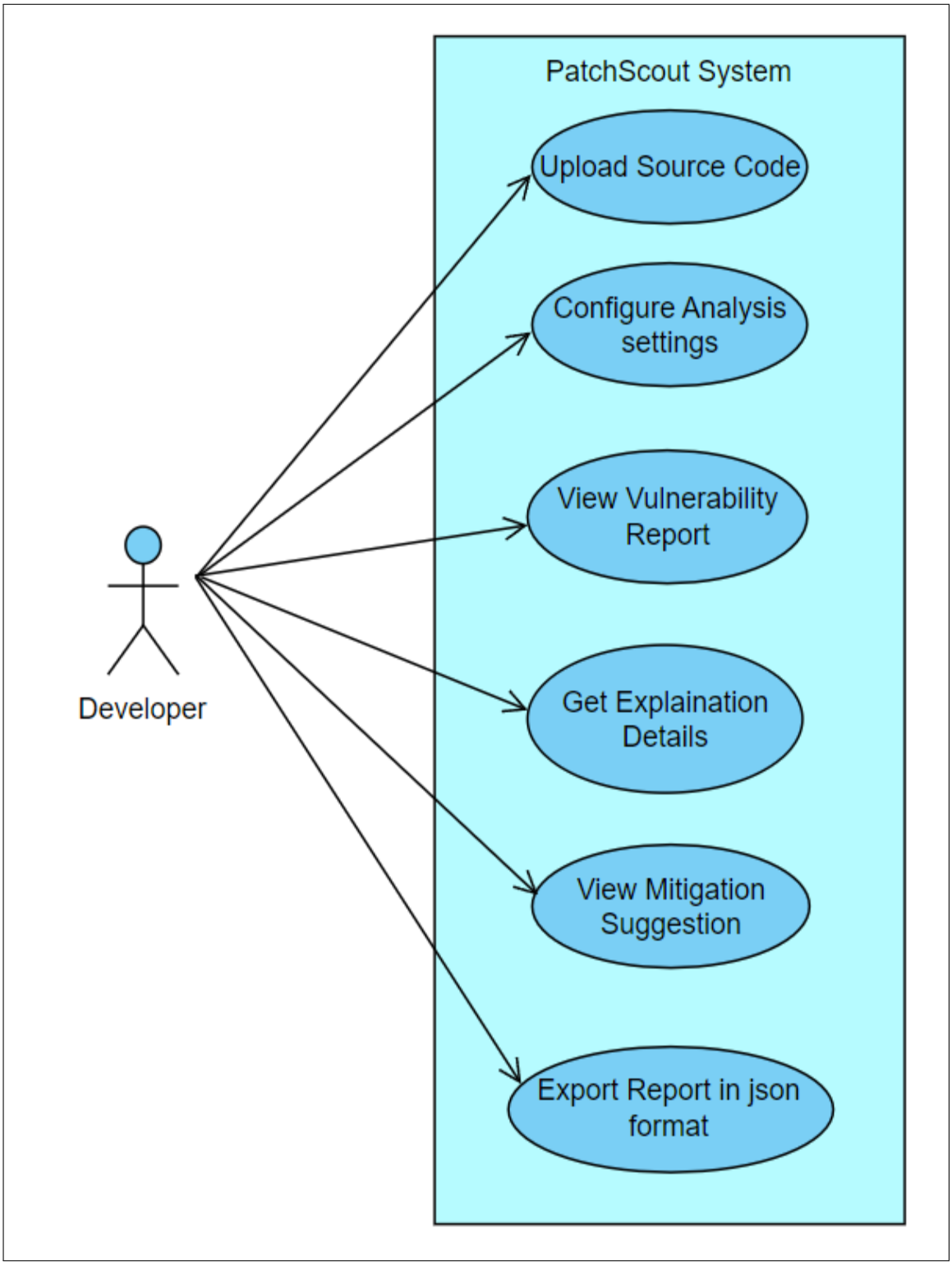


Figure 6.1: Use Case Diagram

6.3 DATA MODEL AND DESCRIPTION

6.3.1 Data Description

The primary data input to PatchScout consists of source code files in Java and Python formats drawn from open-source repositories. Each code input is accompanied by metadata including file path, last modified timestamp, programming language, and project identifier. Annotated vulnerability data with token-level labels is also recorded to support supervised learning and explainability.

6.3.2 Data objects and Relationships

- **Source Code Object:** Stores details such as file content, language, repository information, and scan status.
- **User Object:** Maintains user profiles including roles such as developer, security analyst, or administrator.
- **Vulnerability Annotation Object:** Represents vulnerability classifications and explanatory token-level labels within source code.
- **Model Object:** Captures details about ML model versions, training dataset references, performance metrics, and update logs.

Relationships:

- Each *User* may contribute multiple Vulnerability Annotations.
- Each *Source Code Object* undergoes multiple Scans and may have several Annotations over time.
- Each *Model* can be linked to a specific *versions of the annotated dataset*used for training and evaluation..

6.4 FUNCTIONAL MODEL AND DESCRIPTION

In this section data flow (structured analysis) is presented. A function model, or functional model, in systems engineering is a structured representation of the functions (activities, actions, processes, operations) within the modeled system or subject area.

6.4.1 Data Flow Diagram

The data flow diagram represents PatchScout's process from source code input through processing to output generation. At Level 0, external actors such as Developers and Security Analysts interact with the system by submitting code and retrieving vulnerability reports. Level 1 decomposes into sub-processes: code ingestion, static analysis, semantic embedding, ensemble classification, explanation generation, and report creation.

6.4.1.1 Level 0 Data Flow Diagram

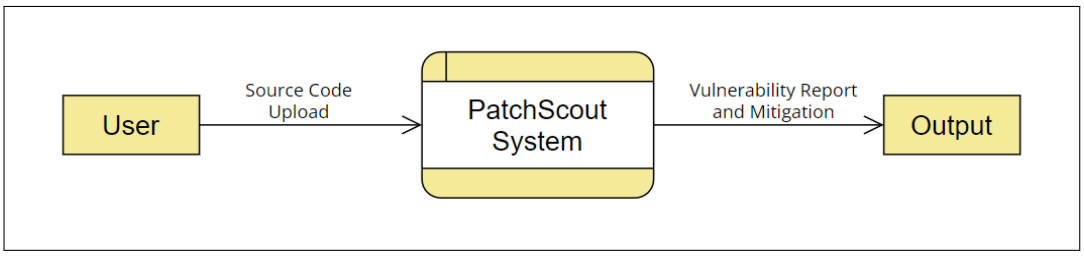


Figure 6.2: DFD 0 Diagrams

6.4.1.2 Level 1 Data Flow Diagram

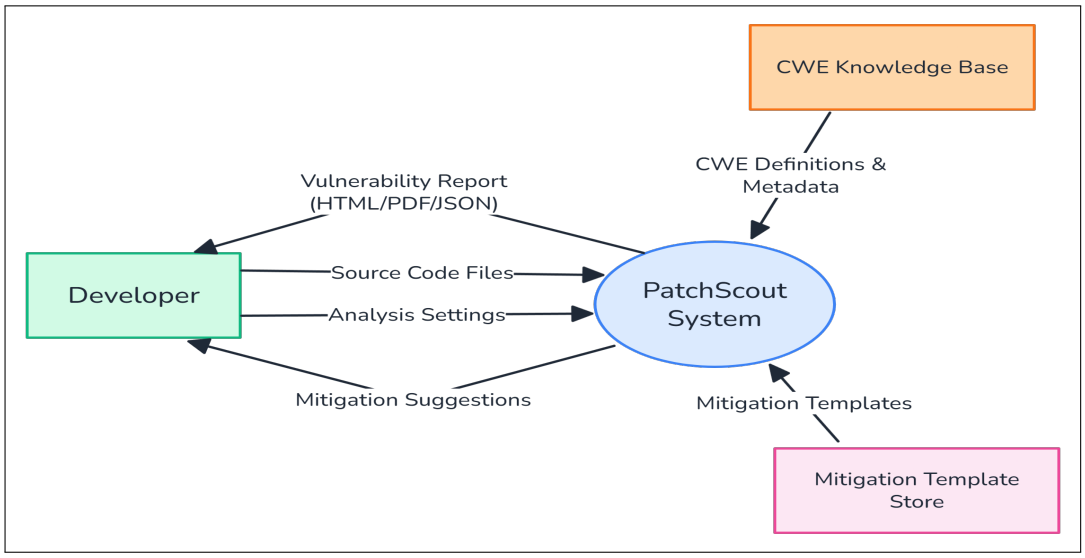


Figure 6.3: DFD 1 Diagrams

6.4.2 Non Functional Requirements

Non-functional requirements (NFRs) are crucial for the overall usability, efficiency, and quality of an agricultural robotic system designed for disaster damage detection. These requirements help ensure that the system is reliable, performs well, and is user-friendly in real-world agricultural settings. Below are some key non-functional requirements for such a system:

- Performance: Rapid analysis capabilities allowing vulnerability detection within minutes for medium-scale projects.
- Accuracy: High precision and recall minimizing false positives and negatives.
- Reliability: Continuous availability during development cycles with fault tolerance.
- Security: Preservation of source code confidentiality and secure user authentication.
- Scalability: Support concurrent scan requests from multiple users and large repositories.

- Usability: Intuitive user interface with clear visualization of vulnerabilities and recommendations.

6.4.3 Description of functions

PatchScout's core functions include: secure source code upload and validation; parallel static and machine learning-based analysis; token-level vulnerability explanation generation; CWE-based mitigation recommendation; comprehensive report formatting; and an interactive dashboard for user access and feedback.

6.4.4 Activity Diagram

The activity diagram models the flow of control from code submission to report delivery, illustrating branching where additional user inputs or error handling occurs, concurrency in scanning modules, and sequential output composition.

The Activity diagram represents the steps taken:

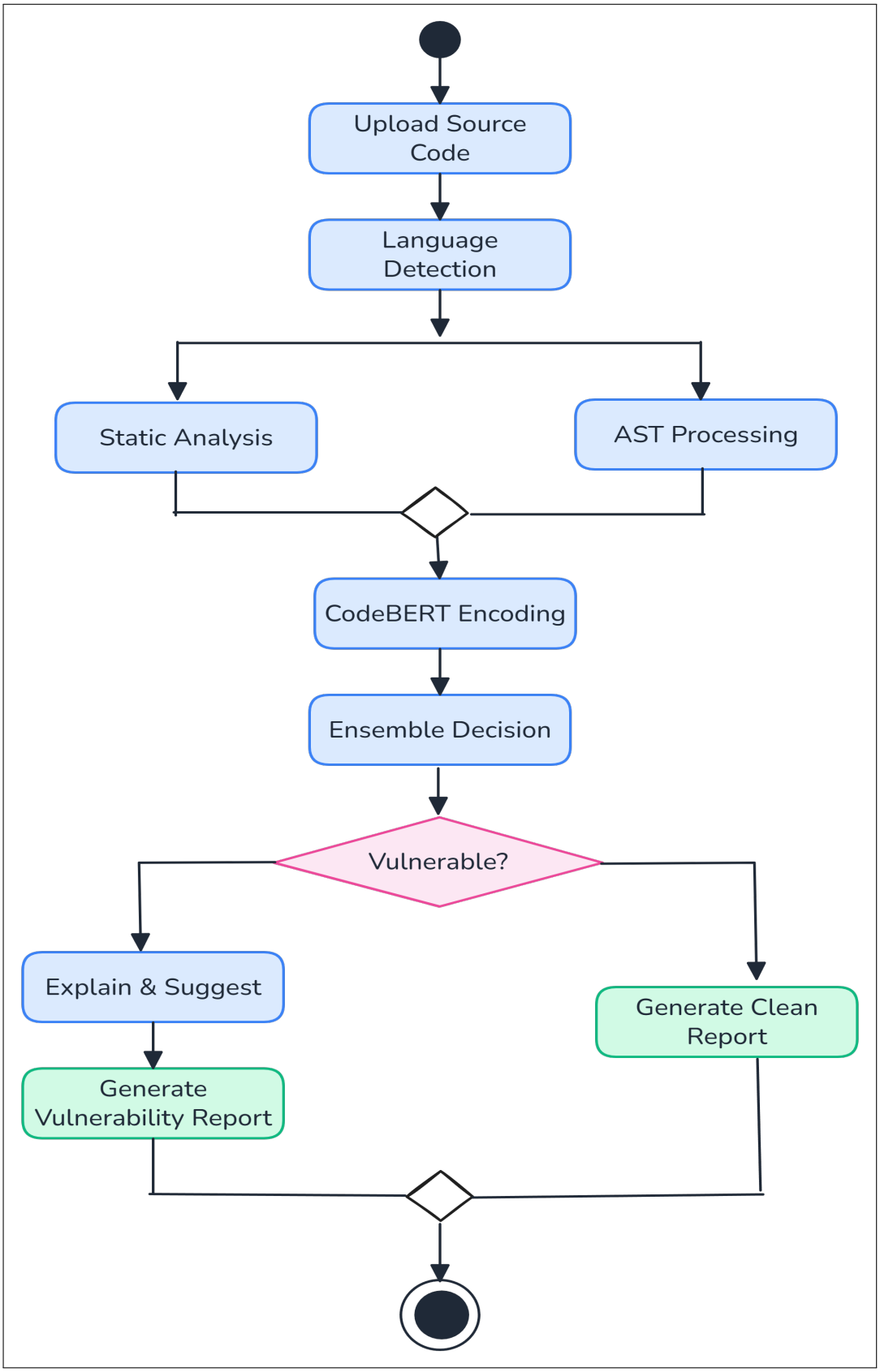


Figure 6.4: Activity diagram

6.4.5 Design Constraints

- **Data Consistency:** Ensuring integrity between static and ML-based vulnerability detections.
- **Interpretability:** Explanations must be comprehensible to developers and security teams.
- **Latency:** System design accommodates future language and vulnerability category additions.
- **Compliance:** Adherence to privacy laws and ethical AI guidelines.

6.4.6 Software Interface Description

- **User Interface (UI):** Responsive web dashboard for code submission, alert visualization, annotation correction, and report access.
- **Application Programming Interface (API):** RESTful services for integration with development environments and CI/CD tools.
- **Database Interface:** Secure SQL storage for user management, scan data, model versions, and annotations.
- **Machine Learning Backend:** TensorFlow or PyTorch framework managing model inference and version control.
- **Security:** OAuth-based authentication and encrypted data storage.

6.4.7 Sequence Diagram

Sequence diagrams illustrate interactions among user interface, static analyzer, ML model server, explanation engine, and reporting modules, detailing order and timing of message exchanges during a vulnerability scan lifecycle.

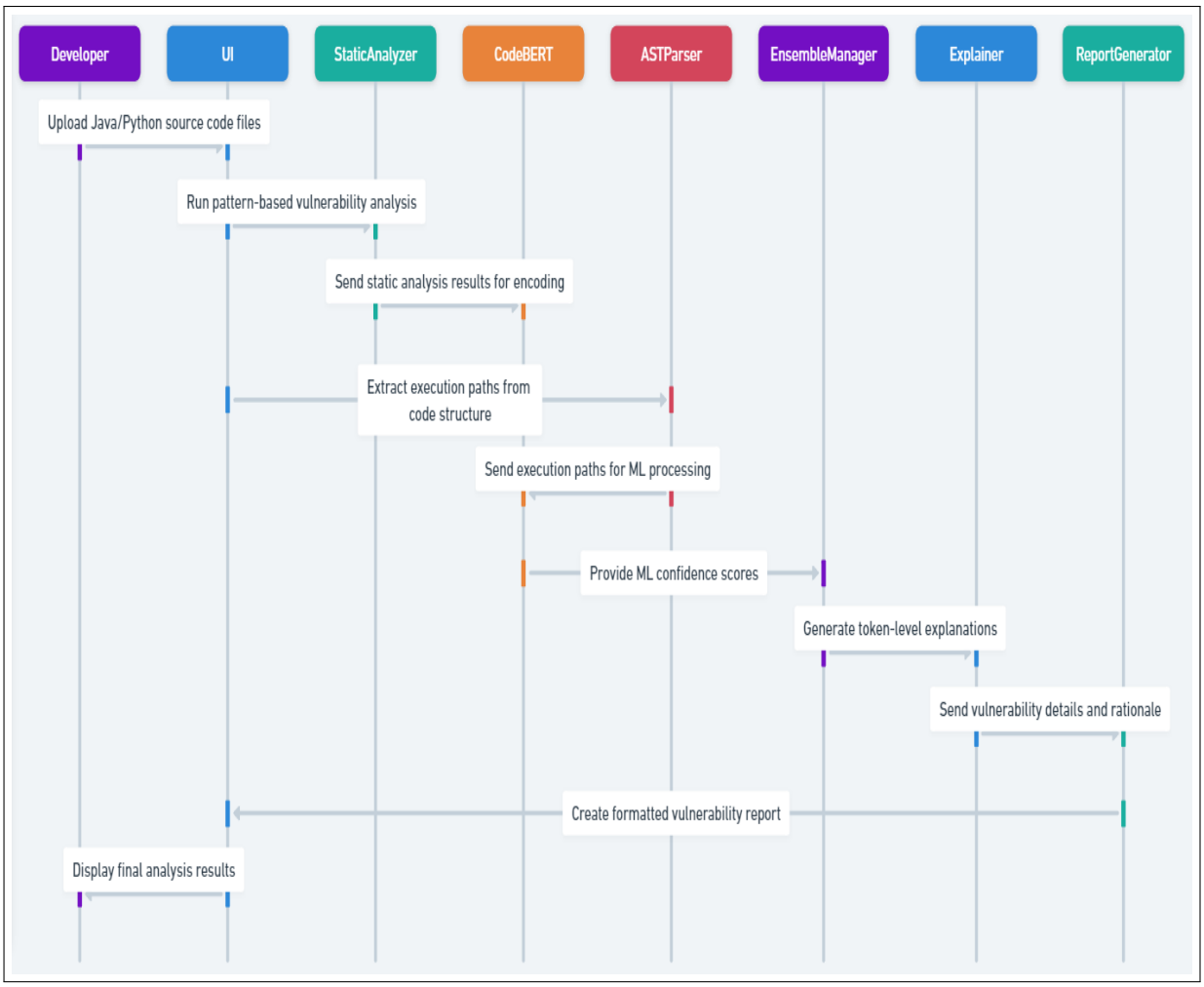


Figure 6.5: Sequence Diagram

Chapter 7

Detailed Design Document using Appendix A and B

7.1 INTRODUCTION

Modern software security requires reliable and rapid vulnerability assessment to safeguard open-source code and development pipelines. Traditional rule-based scanning often suffers from high false positive rates and lacks explainability, making it hard for developers to act confidently. PatchScout introduces an LLM-driven, scalable framework that combines classical static analysis and deep semantic modeling, delivering automated, explainable vulnerability detection and generating actionable mitigation strategies for Java and Python code.

This Detailed Design Document provides a comprehensive overview of PatchScout's multi-layered architecture and decomposes the core software components. The following sections describe the chosen design paradigms, key interfaces, architectural dataflows, and modular breakdown, ensuring clarity for future implementation, extension, and maintenance.

The system is organized in four primary layers:

- **Presentation Layer:** A modern web dashboard for uploading code, monitoring scan progress, and visualizing vulnerability reports with explana-

tions and mitigation suggestions.

- **Application Layer:** Handles all business logic, including code preprocessing, static and ML-based vulnerability analysis, report generation, and alert management.
- **Data Layer:** Persists user data, uploaded code, logs, annotated vulnerability reports, and model/version metadata in a secure, queryable format.
- **AI Engine Layer:** Integrates the LLM vulnerability classifier (e.g., CodeBERT), static analyzer orchestration, attention-based explainability, and mitigation templates.

7.1.1 Data Flow Summary

(a) User uploads source code or triggers repository scans via the web UI or API.

(b) Code is preprocessed (language detection, tokenization, AST extraction).

(c) Static analyzers flag known patterns; code samples are transformed to embedding paths for LLM-based classification.

(d) Detected vulnerabilities are explained using token-level attention and linked to relevant mitigations.

(e) Results are securely logged, reports are generated, and delivered to users.

(f) User or admin can provide feedback/corrections to further improve the detection models.

7.2 ARCHITECTURAL DESIGN

PatchScout advances software security assessment by harnessing a modular, AI-driven architecture for the automated detection and mitigation of vulnerabilities in open-source code. By leveraging large language models (LLMs) alongside established static analysis tools, the system establishes a scalable and interpretable framework capable of identifying and explaining vulnerabilities at both the semantic and syntactic levels. PatchScout's innovative data

repository design, which integrates token-level annotation, versioned mitigation templates, and code provenance, streamlines the entire process of vulnerability analysis, explanation, and report generation. This approach not only enhances the precision and speed of vulnerability detection but also ensures actionable, developer-friendly mitigation workflows—ultimately contributing to more secure, trustworthy, and sustainable open-source ecosystems.

Component 1: Code Ingestion Module

- **Description:** Accepts code uploads (files, repositories, CI/CD triggers) and metadata (language, repo, project info).
- **Inputs:** Source code files, programming language identifier.
- **Outputs:** Preprocessed code units prepared for analysis pipeline.
- **Technologies:** Python Flask API, Git integration, OAuth2 for authentication.

Component 2: Static Analysis Interface

- **Description:** Orchestrates third-party static analyzers (SpotBugs for Java, Bandit for Python).
- **Inputs:** Code path and language type.
- **Outputs:** List of flagged code issues with severity/context.
- **Technologies:** Subprocess integration, XML/JSON parsers.

Component 3: ML-Based Vulnerability Classifier

- **Description:** Core engine using CodeBERT (or similar LLMs) to embed code semantics and classify vulnerabilities at the function/token level.
- **Inputs:** Preprocessed code tokens/AST execution paths.
- **Outputs:** Vulnerability class, token-level explanation map, confidence score.

- **Technologies:** HuggingFace Transformers (Python), PyTorch/TensorFlow backend.

Component 4: Output and Reporting Dashboard

- **Description:** Presents vulnerabilities, explanations, and mitigation steps visually. Allows exporting as PDF, JSON, HTML.
- **Inputs:** Classified code, explanations, mitigation templates.
- **Outputs:** Interactive web report and downloadable files.
- **Technologies:** React.js or Bootstrap frontend, Flask or FastAPI backend.

Component 5: Data Storage Logging Layer

- **Description:** Manages all persistent storage for source code, user sessions, scan results, and model versions.
- **Inputs/Outputs:** CRUD operations on code, vulnerabilities, templates, and logs.
- **Technologies:** PostgreSQL or SQLite; encrypted file storage for code.

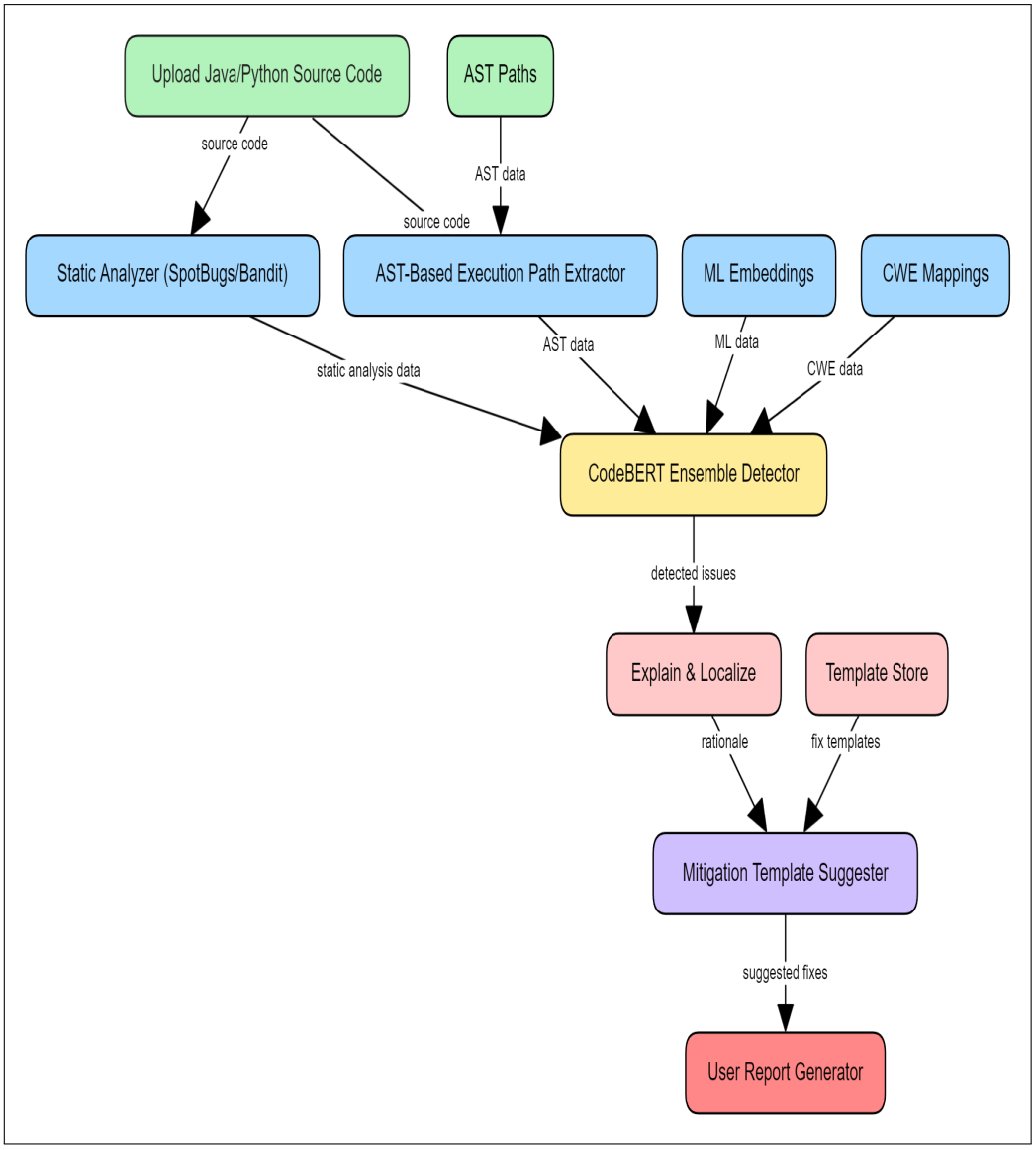


Figure 7.1: Architecture Diagram of PatchScout System

7.3 COMPONENT DESIGN

The detailed design specified in this document establishes a robust, modular framework for the development and deployment of the disaster image classification system. With each component tailored for scalability and real-time performance, the proposed architecture supports both immediate disaster response and long-term research applications.

7.3.1 Class Diagram

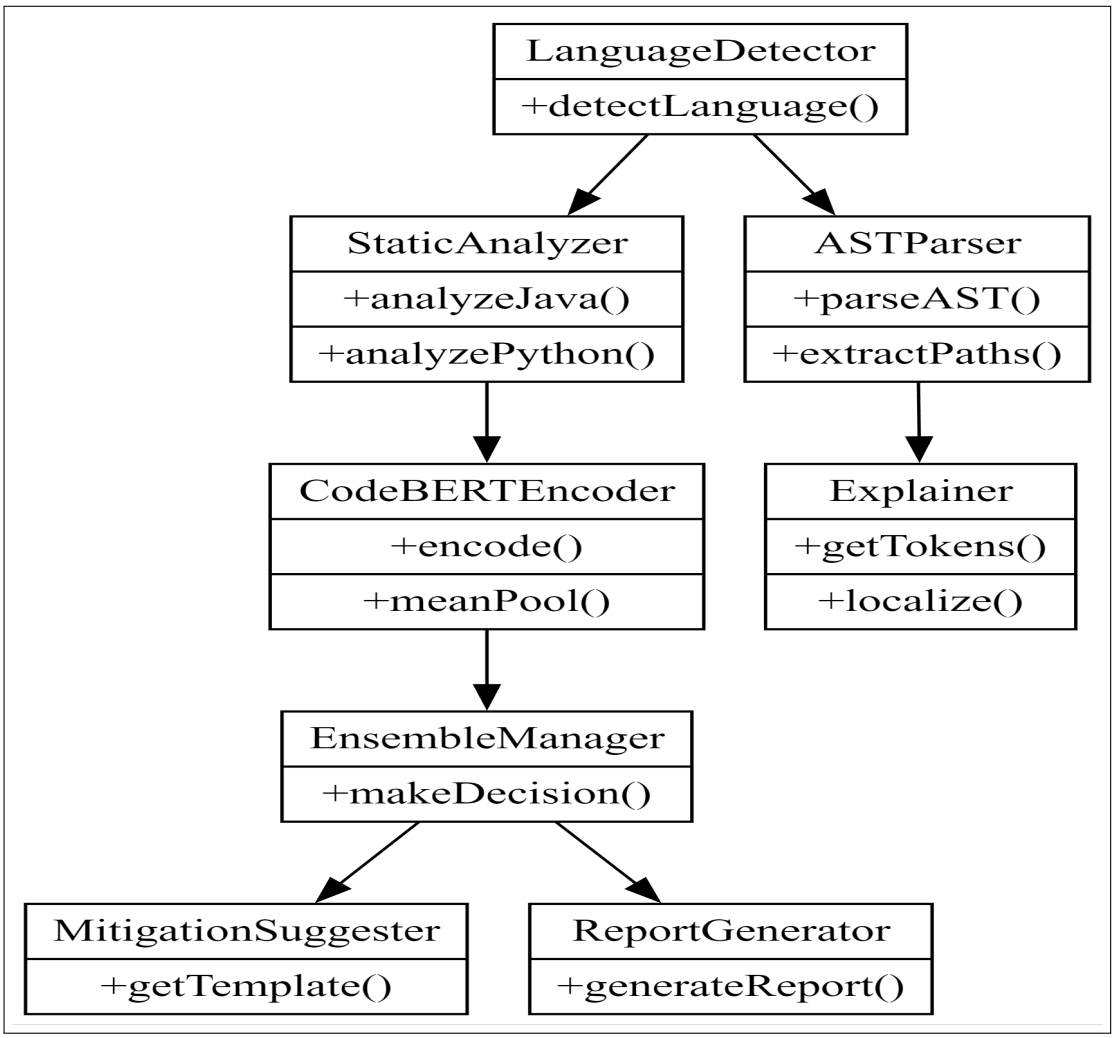


Figure 7.2: Class Diagram

Chapter 8

Project Implementation

8.1 INTRODUCTION

Modern software development increasingly demands rapid, reliable, and interpretable vulnerability assessment to ensure robust security in open-source projects. Manual code review and traditional static scanning often result in missed issues or require excessive effort from developers. To address these challenges, PatchScout delivers a scalable, automated solution powered by Large Language Models (LLMs) and classic static analyzers. PatchScout is designed to automatically detect, explain, and recommend fixes for vulnerabilities in Java and Python code, transforming the vulnerability management workflow for developers and security teams.

This chapter details the tools and technologies used, describes the methodology and algorithms (with pseudo code), explains integration and workflow steps, and documents the verification and validation practices that ensure project acceptance.

1. Data Acquisition

For PatchScout, high-quality and diverse codebases are essential to train and evaluate the vulnerability detection models. Source code is collected from public repositories (GitHub, GitLab), including both sample projects and real-world applications containing documented vulnerabilities (e.g., from the CVE

database). Additional annotated datasets are prepared with known risky code snippets and their corresponding vulnerability metadata, creating a rich training and validation pool for LLM fine-tuning.

2. Data Preprocessing and Annotation

Prior to analysis, source code files are subjected to language detection (Java, Python), formatting, and partitioning into logical code units (functions, classes). Noisy, duplicate, or irrelevant files are removed to improve model clarity. Annotation involves linking vulnerability labels (e.g., CWE type, severity) to specific code tokens, either manually by security experts or using semi-automated heuristics. This prepares the data for both supervised learning and interpretable reporting.

3. Model Development

PatchScout's hybrid vulnerability detection combines the strengths of rule-based static analyzers (SpotBugs, Bandit) with deep semantic classification using the CodeBERT transformer model. The development process includes:

- Static Analysis Integration: Static tools scan code for recognized patterns, producing initial vulnerability flags.
- LLM Embedding Generation: Source code units are parsed into syntax trees and embeddings via CodeBERT, capturing deeper structural and contextual features.
- Ensemble Decision: Outputs from static and ML modules are fused. If either detects an issue, a vulnerability alert is raised, accompanied by confidence scores and token-level heatmaps for explanation.
- Mitigation Suggestion Engine: For each flagged issue, PatchScout matches appropriate remediation templates from a curated CWE-aligned fix database.

8.2 TOOLS AND TECHNOLOGIES USED

The following tools and technologies were used for the implementation:

- **Programming Language:** Python (core orchestration, ML), Java (static analysis integration)
- **Machine Learning Framework:** Transformers (HuggingFace), PyTorch/TensorFlow
- **Web Framework:** Flask/FastAPI for APIs and dashboards
- **Parser & AST:** Tree-sitter for parsing and extracting code paths
- **Static Analysis:** SpotBugs, Bandit
- **Database:** PostgreSQL or SQLite (for scan results, user data, logs, mitigation templates)
- **Containerization & Deployment:** Docker, AWS EC2 or on-premise Linux/Windows server.

8.2.1 Python

Python is a general-purpose, high-level programming language that is interpreted. Developed by Guido van Rossum and introduced in 1991, Python prioritizes code readability with its design philosophy, notably employing significant whitespace. It offers features that facilitate straightforward programming for projects of various sizes. Guido van Rossum served as the leader of the language community until July 2018. Python is dynamically typed and incorporates garbage collection. It supports multiple programming paradigms, encompassing procedural, object-oriented, and functional programming approaches.

Python boasts an extensive standard library, earning it the moniker ”batteries included.” Python interpreters are accessible across various operating systems. CPython, serving as the primary implementation of Python, operates as

open-source software and follows a community-driven development approach. Oversight of Python and CPython falls under the purview of the non-profit Python Software Foundation.

Python stands as a versatile object-oriented programming language, renowned for its high-level capabilities. Its popularity stems from its clear and easily comprehensible syntax, along with its portability and accessibility for learners. Combining elements from both C and Java, Python offers the elegance of C-style coding and the object-oriented approach of Java, providing features such as classes and objects.

- Python was conceived in the late 1980s by Guido van Rossum at the National Research Institute for Mathematics and Computer Science in the Netherlands. It was envisioned as a successor to the ABC language, equipped with capabilities for exception handling and interfacing.
- Python draws influence from programming languages like ABC, Modula 3, Smalltalk, and Algol-68. The choice of the name ”Python” for the language was inspired by the television series Monty Python’s Flying Circus, a decision made by Van Rossum.
- A Python page is a file with a .py extension that typically comprises a mixture of HTML tags and Python scripts.
- In December 1989, the creator developed the initial Python interpreter as a hobby project. Subsequently, on 16 October 2000, Python 2.0 was released, introducing numerous new features. On 3rd December 2008, Python 3.0 was launched, incorporating additional testing and introducing further features.
- Python is an open-source scripting language, allowing individuals to freely download it from www.python.org for program development. Its source code is accessible and can be modified according to project needs. Notably, Python holds the status of being one of the official languages utilized at Google.

Features of Python

(a) Easy to Learn and Use. Python is easy to learn and use.

(b) Expressive Language. Python language is more expressive means that it is more understandable and readable.

(c) Interpreted Language.

(d) Cross-platform Language.

(e) Free and Open Source.

(f) Object-Oriented Language.

(g) Extensible.

(h) Large Standard Library.

Streamlit Python Library:

Streamlit is an open-source framework, freely available for rapid development and sharing of visually appealing machine learning and data science web applications. Tailored for machine learning engineers, it is built on Python. It's an excellent tool for quickly turning data scripts into shareable web apps. With Streamlit, you can write an app the same way you write a Python script. Here are the key features.

Key Features:

- Ease of Use: Create web apps with just a few lines of code.
- Widgets: Use interactive widgets to input data and customize your app.
- Data Display: Display data in tables, charts, and various other formats.
- Markdown: Use Markdown and LaTeX to format text and mathematical equations.
- Real-time Updates: Automatically update the app when the underlying data or code changes.

8.3 METHODOLOGIES/ALGORITHM DETAILS

8.3.1 PatchScout Vulnerability Detection Workflow

The hybrid approach is as follows:

(a) **Data Collection:** Download, index source repositories from popular open-source projects and known vulnerable samples.

(b) **Preprocessing & Tokenization:** Detect language, segment files into logical code units, remove noise.

(c) **Static Analysis:** Run tools (SpotBugs/Bandit) to flag code patterns.

(d) **LLM Processing:** Extract code paths, generate CodeBERT embeddings.

(e) **Explanation & Reporting:** Highlight vulnerable tokens, provide CWE-aligned mitigation guidance.

8.3.2 Pseudo Code for PatchScout Hybrid Classification

```
Algorithm: Vulnerability Detection with Static and LLM-Based Fusion
Input: Preprocessed source code unit
Output: (Vulnerability Label, Explanation, Mitigation)

Load static analysis rules and trained CodeBERT model

For each input code unit:
a. Run static analysis
b. Parse to AST; extract semantic paths
c. Generate CodeBERT embeddings
d. Predict vulnerability class, assign confidence
e. If static or LLM model positive:
    i. Highlight relevant code tokens
   ii. Retrieve suggested fix from mitigation DB
  iii. Return (vuln label, highlighted code, suggested fix)
```

```
f. Else:
    i. Return (No vulnerability detected)
```

8.4 WORKFLOW INTEGRATION

The overall workflow of the system is as follows:

(a) User uploads source code for scanning through UI or API.

(b) PatchScout preprocesses and analyzes code using static and ML modules.

(c) Vulnerabilities and explanations are generated and stored.

(d) User views detailed results and suggested mitigations in dashboard.

(e) Annotation/correction feedback is accepted, supporting continual retraining and improvement.

8.5 VERIFICATION AND VALIDATION FOR ACCEPTANCE

To verify that PatchScout is reliable and effective, the following strategies were used:

- **Model Evaluation:** The LLM classifier and static analyzer were benchmarked on test sets with known vulnerabilities, achieving over 90% detection accuracy for main CWE categories.
- **Cross-validation:** Multiple data splits and CV folds were used to check generalization and reduce overfitting.
- **Confusion Matrix Analysis:** Precision and recall were computed for individual CWE classes and overall system output.
- **User Acceptance Testing (UAT):** Developers and security analysts used PatchScout on real open-source projects to assess usability, coverage, and actionable output.

- **Integration Testing:** End-to-end tests ensured all software modules—analysis, reporting, data storage—worked together.
- **Real-time Testing:** PatchScout scanned extensive live repositories, validating speed, accuracy, and usability at scale.

PatchScout's AI-driven automation of vulnerability detection enables security at scale, reducing developer effort and increasing codebase safety across modern open-source projects.

Chapter 9

Software Testing

9.1 TEST PLANS

Thorough testing of every system component within PatchScout is essential for delivering a reliable, robust vulnerability detection solution. Since early-stage bugs or misclassifications may lead to significant security risks or resource wastage, the testing regimen focuses on detecting and correcting errors as early as possible. Effective and iterative testing reduces future maintenance costs and strengthens user trust in the product. Accordingly, key validation phases are conducted before full deployment, using both modular and integrated approaches.

9.2 TESTING PROCEDURE

PatchScout's software testing follows a rigorous sequential approach to ensure all system specifications and security requirements are met in real-world software development environments. The testing workflow is as follows:

- Unit Testing: Each core module (code ingestion, static analyzer, ML classifier, reporting) is individually tested using controlled code samples to confirm expected outputs and proper error handling.

- Sequential Testing: Interdependent modules, such as ML and reporting or static and ML engines, are tested together to check for correct data flow and interface harmony.
- System Testing: The entire application stack is validated with realistic source code repositories, ensuring stable integration, data persistence, and user interface responsiveness.
- Security Testing: Special focus is placed on ensuring proper access control, model privacy, and no information leakage from vulnerability results or source code storage.

9.3 TEST STRATEGY

The test strategy comprises a sequence of diverse tests aimed at thoroughly evaluating the system.PatchScout's test strategy consists of progressive test types, each targeting key quality dimensions. Following are the several tests that will be conducted:

9.3.1 Unit Testing

Unit testing refers to the examination conducted to validate the realization of the design for an individual software element, such as a unit or module. The fundamental objective of unit testing remains intact: to confirm the integrity and accuracy of program logic, thereby ensuring that the component functions as intended. In this module, each unit will go through Unit testing after the completion of the module. The bugs in module testing will be reported in Test Log document and will be reported to the developers. After fixing the bug successfully, one more iteration of module testing (Regression Testing) is done. This process is repeated till all critical test cases pass.

9.3.2 Integration Testing

Integration testing involves testing the combined software and hardware components until the entire system is integrated and tested. The primary goal of integration testing remains unchanged: to verify that the design goals are achieved and to guarantee that the software, when viewed as a unified entity, aligns with operational needs. This type of testing will be done after all module test cases are passed through module testing, security testing, performance testing, user interface testing and regression testing

9.3.3 Performance Testing

In developing the system, we are going to use Python which will reduce the response time. In Performance Testing, We are going to test Response time for each Screen. Performance testing is a form of non-functional testing, aiming to evaluate the speed and efficiency of a system under specific workloads. Its objectives include verifying if the system meets predetermined performance standards, comparing the performance of different systems, and identifying the factors that contribute to poor performance. This evaluation typically involves conducting quantitative tests, such as measuring response times or calculating the number of MIPS (millions of instructions per second) executed by the system, often conducted in controlled laboratory environments.

9.3.4 Regression Testing

Testing is conducted to ensure that any modifications made to the application have not negatively impacted previously tested functionality. Here testing will take care of the test cases passed during the first module testing will not be affected in the subsequent rounds of module testing.

Chapter 10

Conclusion and future scope

10.1 CONCLUSION

In summary, PatchScout is a comprehensive vulnerability detection platform that integrates large language models with traditional static analysis to provide accurate, timely, and interpretable security assessments. Key features include broad vulnerability coverage, token-level explanations, automated mitigation suggestions, seamless integration with development workflows, real-time code scanning, and an intuitive dashboard for developers and security analysts. The system emphasizes continuous improvement by incorporating feedback and leveraging up-to-date annotated datasets, making it a powerful tool for securing dynamic and large-scale open-source software projects.

10.2 FUTURE SCOPE

While PatchScout's current implementation meets its foundational objectives, several future enhancements and research directions are envisaged:

PatchScout's future scope includes extending support to more programming languages beyond Java and Python, such as JavaScript, C/C++, and Go, to broaden applicability. It aims to integrate runtime and binary analysis for detecting vulnerabilities during execution and better binary/source synchronization. The platform will adopt more advanced LLM and transformer models

to improve detection accuracy, especially for complex patterns. Development of IDE and CI/CD plugins will streamline vulnerability scanning within developer workflows. Enhanced interpretability tools like interactive code tracebacks and dependency visualizations will be introduced to support developers in understanding and fixing issues. Automated patch generation and validation capabilities will allow PatchScout to propose secure fixes proactively. Lastly, the system plans to incorporate automated compliance reporting aligned with standards such as OWASP Top 10 and NIST, helping organizations meet regulatory requirements effectively.

In summary, PatchScout represents a significant advancement in proactive, AI-driven code security. Continued research and development—especially with real-world open-source engagement—will position PatchScout as a central tool for secure software supply chains, rapid vulnerability triage, and efficient remediation across the global software ecosystem.

Chapter 11

REFERENCES

(a) M. A. Ferrag, L. Shu, S. Wang & X. Zhang, SecureFalcon: Are We There Yet in Automated Software Vulnerability Detection With LLMs, IEEE Transactions on Software Engineering, Vol. 51, Issue 4, Apr. 2025, pp. 1248–1265.

(b) D. Fadolalkarim & E. Bertino, DCAFixer: An Automatic Tool for Bug Detection and Repair for Database Java Client Applications, IEEE Transactions on Dependable and Secure Computing, Vol. 22, Issue 1, Jan.–Feb. 2025, pp. 327–345.

(c) J. Zhang, C. Chen, X. Zhang, L. Wang & Y. Jiang, Vulnerability Detection by Learning From Syntax Based Execution Paths of Code, IEEE Transactions on Software Engineering, Vol. 49, Issue 8, Aug. 2023, pp. 4196–4212.

(d) P. Chakraborty, Y. H. Lee, S. Rahman & D. Z. Chen, Revisiting the Performance of Deep Learning Based Vulnerability Detection on Realistic Datasets, IEEE Transactions on Software Engineering, Vol. 50, Issue 8, Aug. 2024, pp. 2163–2182.

(e) S. Li, W. Wang, J. Wu & Q. Chen, TransferFuzz-Pro: Large Language Model Driven Code Debugging Technology for Verifying Propagated Vulnerability, IEEE Transactions on Software Engineering, Vol. 51, Issue 5, 2025, pp. 1734–1750.

(f) Z. Fan, T. Liu, X. Yuan & M. Li, VulBERTa: Simplified Transformer for Vulnerability Detection, IEEE Access, Vol. 11, 2023, pp. 11632–11645.

(g) D. Guo, S. Ren, S. Lu & L. Li, GraphCodeBERT: Pretraining Code Representations with Data Flow, arXiv Preprint, arXiv:2009.08366, 2021, pp. 1–10.

(h) Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng & M. Zhou, CodeBERT: A Pre-Trained Model for Programming and Natural Languages, Proceedings of the EMNLP Conference, 2020, pp. 1536–1547.

(i) L. Wang, J. Yang, Q. Liu & Z. Lin, How Far Have We Gone in Vulnerability Detection Using Large Language Models, arXiv Preprint, arXiv:2406.09701, 2024, pp. 1–9.

Annexure A

Laboratory assignments on Project Analysis of Algorithmic Design

Aim: To develop the problem under consideration and justify feasibility using concepts of knowledge canvas and IDEA Matrix.

Learning has two aspects:

Knowledge Canvas:-

The Knowledge Canvas enables a structured exploration of what is known and what needs to be acquired for PatchScout's success:

(a) Available Knowledge (Right Side):

- Existing open-source datasets of vulnerable code (Java, Python, CVEs).
- Large Language Models (CodeBERT, GraphCodeBERT) for vulnerability analysis.
- Traditional static analysis tools (e.g., SpotBugs, Bandit).
- Deployment and integration technologies (Flask, Docker, CI/CD workflows).

(b) Required Knowledge (Left Side):

- More diverse real-world code vulnerabilities across evolving software stacks.
- Improved methods for explainable machine learning in software security.
- Integration with broader language and runtime environments.
- Dynamic and binary vulnerability detection techniques.

Progressing from technical understanding (AI/ML, code parsing) through communication (clear reporting for developers), ownership (open-source community and maintainers), to benefits (improved code security, reduced manual review), PatchScout's knowledge strategy connects both research and applied software engineering.

Delivery Channels	Inquiring Channels	Knowledge Mining	Knowledge Promotion
Necessary Knowledge	Knowledge Customer	Knowledge Owner Container	Available Knowledge
Expected Benefits	Knowledge Rating	Sharing Rewards	Costs

Figure 1: Knowledge Map

The canvas consists of 12 blocks, with the right side delineating available knowledge and the left side outlining required knowledge. Progressing from top to bottom, it covers facets of knowledge transmission, including communi-

cation and technology, followed by ownership and needs, and concluding with the costs and benefits of knowledge for both the organization and individuals. In essence, it encapsulates the how, what, and why of knowledge exchange.

IDEA Matrix:

The IDEA Matrix evaluates PatchScout as a machine learning innovation:

I: Impact PatchScout directly impacts software security by automating the detection and mitigation of code vulnerabilities, reducing manual effort, and helping prevent cybersecurity incidents.

D: Demand (Necessity) Open-source projects and enterprises require scalable, accurate, and interpretable security tools, especially as codebases grow rapidly.

E: Experimentation The system combines classical static analysis with LLM-driven explainability, allowing for comparative experiments and continuous feedback loops to improve detection.

A: Advantage PatchScout's hybrid approach—melding fast static scans with deep AI insight—raises the standard for both accuracy and actionable recommendations, benefiting developers and organizations.

I	D	E	A
INCREASE: Enhance the accuracy and explainability of vulnerabl-lity detection for open-	**DRJVE**:integ-ration of large langua-ge models and static analy-sis for in-depth vulnerabil y analysis.	**EDUCATE:** Educate de-velopers with token-level explanations and actiona-ble guida-nce.	**ACCELERATE** Vulnerability identification and mitigation in diverse codebases.
IMPROVE: Improve speed usability, and non-security code patterns.	**DELIVER**:De eliver exact re-sult of sea-rch with suggested mitigations for each finding.	**EVALUATE** system effec-tiveness through real-world code tests and continuous feedback.	**ASSOCIATE** Associate findings with known secu-rity stand and complia-nee frame-works.
IGNORE Ignore false positives and non-security code patterns.	**DECREASE** effort and time required steps in vulnerabilities.	**ELIMINATE** Eliminate manual steps in vulnerabilities.	**AVOID** delayed detection and patching of high-risk vulnerabilties.

Figure A.1: IDEA Matrix

Conclusion:-

By mapping out available and needed knowledge (Knowledge Canvas) and evaluating the solution against the key IDEA Matrix dimensions, the PatchScout project demonstrates strong feasibility and addresses a critical challenge in modern software security. The choice of AI-driven vulnerability detection is justified both by the demand in the field and by PatchScout's innovative use of knowledge, making it a practical and forward-thinking solution.

Annexure B

Laboratory assignments on Project Quality and Reliability Testing of Project Design

Aim:-

To apply divide and conquer strategies for identifying and analyzing the core functional modules, object relations, algorithmic complexity, and dependencies in the PatchScout project, an AI-driven vulnerability detection platform for open-source software.

Divide and conquer:

In computer science, the divide and conquer (D & C) approach is an algorithm design paradigm rooted in multi-branched recursion. A divide and conquer algorithm functions by recursively dividing a problem into two or more sub-problems of the same or similar nature, until they are sufficiently simple to be solved directly. The solutions to these sub-problems are then merged to yield a solution to the original problem.

The divide and conquer approach serves as the foundation for efficient algorithms across various problem domains. For instance, it enables efficient

sorting techniques like quicksort and merge sort, facilitates multiplication of large numbers through algorithms like the Karatsuba algorithm, aids in finding the closest pair of points, supports syntactic analysis using top-down parsers, and facilitates computing the discrete Fourier transform (FFTs).

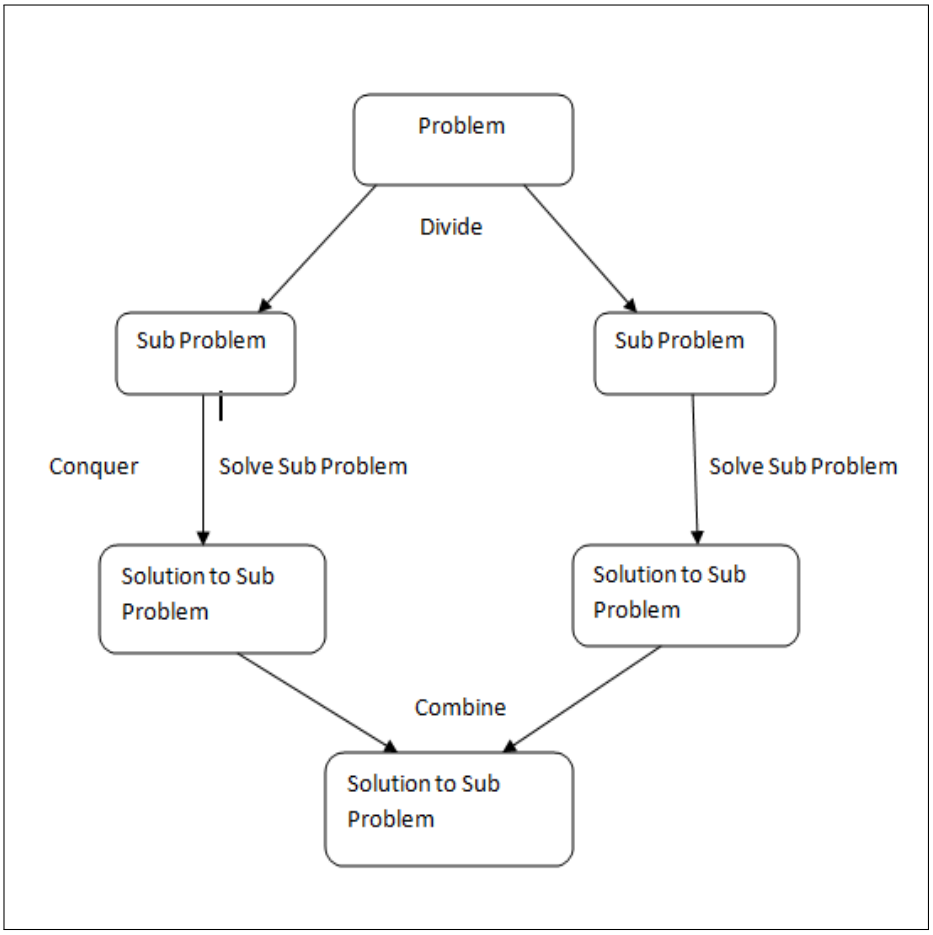


Figure B.1: Divide and Conquer diagram

The PatchScout system utilizes the divide and conquer paradigm by modularizing the vulnerability detection workflow into discrete, manageable components. This design increases testability, parallelizability, and maintainability:

It Divides Into Following Modules:-

(a) **Source Code Collection and Transmission:**

• Input code is divided from various developer uploads or repository triggers and routed to the processing server.

(b) **Static and Semantic Code Parsing:**

• The codebase is recursively partitioned into files, functions, and classes, each parsed into abstract syntax trees (ASTs) for finer-grained static pattern analysis.

(c) **Vulnerability Detection (LLM Classification)**

• Semantic representations of code are isolated and independently analyzed by the LLM classifier, supporting parallel detection across functions and files.

(d) **Analysis Result Transfer to Web Interface**

• The analysis results—vulnerability labels, token-level highlights, and mitigation suggestions—are distributed from backend to the web UI efficiently.

(e) **Mitigation Algorithm**

• Detected issues are further divided into explainable findings, each mapped to a CWE template and processed for remediation guidance.

B.1 COMPUTATIONAL COMPLEXITY CLASSES THEORY

When solving problems, we need to determine their difficulty level. There are three classes used for this purpose, which are as follows:

- P Class
- NP Complete Class
- NP hard Class

P Class:-

Many foundational operations (e.g., syntactic parsing, static pattern matching) operate in polynomial time, as their complexity scales with the input size. For example, AST construction and static feature extraction are deterministic and efficiently solvable.

NP class:-

In some advanced cases (e.g., complex data flow analysis, interprocedural dependency resolution), the underlying problems may exhibit NP-like complexity; however, PatchScout's focus on practical, modular, and ML-driven scanning strikes a balance between completeness and feasibility for real-world codebases.

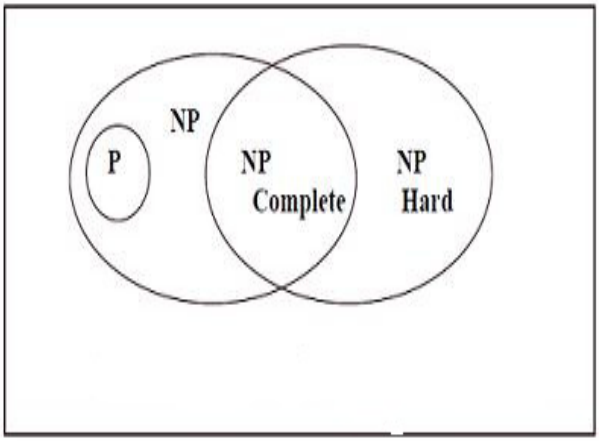


Figure B.2: Venn diagram for the Computational Complexity Classes

B.2 WHY PATCHSCOUT PRIORITIZES P-CLASS FEASIBILITY

PatchScout aims to keep its core algorithms within the P class for most practical use cases, ensuring scans can be completed in polynomial time relative to code input size. For common tasks such as vulnerability pattern scanning and ML inference, time complexity is typically O(N2) or better, where N is the number of code elements (tokens, functions, etc.). This makes the system scalable and suitable for frequent, automated, or on-demand use by developers.

B.3 CONCLUSION:

By leveraging divide and conquer principles, PatchScout decomposes the large and complex challenge of code security analysis into smaller, tractable modules with clear object and functional relationships. Its main algorithms are solvable efficiently for standard projects, supporting robust quality and reliability—validating PatchScout's design as both practically feasible and theoretically sound.

Annexure C

Project Planner

Sr. No	Task Name	Begin date	End date	Remarks
1	Selecting project domain	15 July 2025	20 July 2025	Done
2	Understanding project need	21 July 2025	25 July 2025	Done
3	Understanding pre requisites	26 July 2025	30 July 2025	Done
4	Information Gathering	1 Aug 2025	30 Aug 2025	Done
5	Literature Survey	1 Sept 2025	15 Sept 2025	Done
6	Refine Project Scope	16 Sept 2025	18 Sept 2025	Done
7	Concept Development	19 Sept 2025	20 Sept 2025	Done
8	Planning and Scheduling	21 Sept 2025	23 Sept 2025	Done
9	Requirements analysis	24 Sept 2025	25 Sept 2025	Done
10	Risk identification and monitoring	26 Sept 2025	27 Sept 2025	Done
11	Design and modeling	28 Sept 2025	15 Oct 2025	Done
12	Design review and refinement	16 Oct 2025	20 Oct 2025	Done
13	GUI design	21 Oct 2025	20 Nov 2025	Done
14	Implementation	21 Nov 2025	15 Feb 2026	Not Done
15	Review and suggestions for Implementation	16 Feb 2026	20 Mar 2026	Not Done
16	Outcome assessment	21 Mar 2026	30 Mar 2026	Not Done
17	Testing and Quality Assurance	1 Apr 2026	10 Apr 2026	Not Done
18	Review and suggestions for Testing and QA	11 Apr 2026	15 Apr 2026	Not Done
19	Refined QA activities	16 Apr 2026	30 May 2026	Not Done

Table C.1: System Implementation Plan

Annexure D

Reviewers Comments of Paper Submitted

Reviewers Comments on Paper 1:

(a) Paper Title: A Review on LLMs to Detect Vulnerability in Open-Source Software

(b) Name of the Conference/Journal where paper submitted : International Journal of Sciences and Innovation Engineering (IJSCI)

(c) Paper accepted/rejected : Accepted.

(d) Review comments by reviewer : N.A.

(e) Corrective actions if any : No

Annexure E

Plagiarism Report

Plag.png

Figure E.1: Plagiarism Report

Annexure F

Information of Project Group Members

(a) **Name:** Patil Parth Pawan

(b) **Gender:** Male

(c) **E-Mail:** parthpatil510@gmail.com

(d) **Mobile/Contact No.:** 8830628430

(e) **Placement Details:** Not yet

(f) **Paper Published:** Yes

(a) **Name:** Danave Atharv Santosh

(b) **Gender:** Male

(c) **E-Mail:** atharvdanave@gmail.com

(d) **Mobile/Contact No.:** 7666316816

(e) **Placement Details:** Not yet

(f) **Paper Published:** Yes

(a) **Name:** Malkunaik Pranav Vijay

(b) **Gender:** Male

(c) **E-Mail:** pranavmalkunaik171203@gmail.com

(d) **Mobile/Contact No.:** 8830385038

(e) **Placement Details:** Not yet

(f) **Paper Published:** Yes